

ASSIGNMENT-1

Write a C program to demonstrate client server architecture.

PROGRAM :-

Server.c

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<unistd.h>
#include<arpa/inet.h>

int main()
{
    int serverSocket, clientSocket;
    char buffer[100];
    struct sockaddr_in serverAddr;
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    bind(serverSocket, (struct sockaddr*)&serverAddr,
    sizeof(serverAddr));
    listen(serverSocket, 5);
    printf("Server waiting for Connection .... \n");
    clientSocket = accept(serverSocket, NULL, NULL);
    read(clientSocket, buffer, sizeof(buffer));
    printf("Client says: %s \n", buffer);
    char msg[] = "Hello from Server!";
    write(clientSocket, msg, strlen(msg)+1);
    close(serverSocket);
    close(clientSocket);
    return 0;
}
```


Client.c

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<unistd.h>
#include<arpa/inet.h>

int main()
{
    int clientSocket;
    char buffer[100];
    struct sockaddr_in serverAddr;
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    connect(clientSocket, (struct sockaddr *)&serverAddr,
            sizeof(serverAddr));
    char msg[] = "Hello from Client!";
    write(clientSocket, msg, strlen(msg)+1);
    read(clientSocket, buffer, sizeof(buffer));
    printf("Server says: %s\n", buffer);
    close(clientSocket);
    return 0;
}
```

OUTPUT:-

Server Terminal Output

Server waiting for connection...

Client says: Hello from client!

Client Terminal Output

Server says: Hello from server!

ASSIGNMENT-2

Write a C program to check the data error using checksum.

PROGRAM:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef unsigned char DataSegment;
DataSegment calculate_checksum(const DataSegment *data,
int size)
{
    unsigned int sum = 0;
    int i;
    for(i = 0; i < size; i++)
    {
        sum += data[i];
    }
    unsigned char eight_bit_sum = (unsigned char)(sum &
0xff);
    DataSegment checksum = (DataSegment)(~ eight_bit_sum);
    return checksum;
}
int verify_checksum(const DataSegment *data, int size) {
    unsigned int sum = 0;
    int i;
    for(i = 0; i < size; i++)
    {
        sum += data[i];
    }
    unsigned char eight_bit_sum = (unsigned char)(sum & 0xff);
    unsigned char complement_sum = (unsigned char)(~ eight_bit
-sum);
    return (complement_sum == 0)
}
```



```

int main()
{
    DataSegment data[] = {0x0A, 0x14, 0x05, 0x02};
    int data-size = sizeof(data) / sizeof(DataSegment);
    printf("--- Checksum Data Integrity check --- \n\n");
    printf("Original Data Segments : 0x%.02X, 0x%.02X, 0x%.02X, 0x%.02X\n", data[0], data[1], data[2], data[3]);

    DataSegment checksum_tx = calculate_checksum(data, data-size);
    printf("\n Calculated Checksum : 0x%.02X\n", checksum_tx);

    DataSegment received_msg_correct[data-size + 1];
    memcpy(received_msg_correct, data, data-size);
    received_msg_correct[data-size] = checksum_tx;
    printf("Received Message : 0x%.02X, 0x%.02X, 0x%.02X, 0x%.02X, 0x%.02X\n", received_msg_correct[0], received_msg_correct[1], received_msg_correct[2], received_msg_correct[3], received_msg_correct[4]);

    int check_result_correct = verify_checksum(received_msg_correct, data-size + 1);
    printf("Verification Result: \n");
    if(check_result_correct)
    {
        printf("SUCCESS : Data is error free\n");
    }
    else { printf("ERROR: Data Error Detected\n"); }
    return 0;
}

```

OUTPUT:-

```

--- Checksum Data Integrity check ---
Original Data Segments : 0x0A, 0x14, 0x05, 0x02
Calculated Checksum : 0xD5
Received Message : 0x0A, 0x14, 0x05, 0x02, 0xD5
Verification Result : SUCCESS : Data is Error-free

```

Baisakhi Behara
Section - A
Roll No - 2505014

ASSIGNMENT-3

Write a C program for performing Selective Repeat ARQ.

PROGRAM :-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define WINDOW_SIZE 4
#define MAX_SEQ (2 * WINDOW_SIZE)
#define MAX_PACKETS 10
#define TIMEOUT 5

typedef struct {
    int seq_num;
    char data[30];
} Packet;

typedef struct {
    int seq_num;
    int ack_type;
} Ackframe;

typedef struct {
    Packet buffer[MAX_SEQ];
    int next_seq_to_send;
    int window_start;
    int sent_count;
    int is_acked[MAX_SEQ];
    int timers[MAX_SEQ];
} Sender;

typedef struct {
    Packet buffer[MAX_SEQ];
    int expected_seq_num;
    int received_flags[MAX_SEQ];
} Receiver;
```



```
int is_lost(int probability_percent)
```

```
{ if(rand() % 100 < probability_percent)
```

```
{ return 1;
```

```
} return 0;
```

```
}
```

```
void sender_init(Sender *s){
```

```
s->next_seq_to_send = 0;
```

```
s->window_start = 0;
```

```
s->sent_count = 0;
```

```
for(int i = 0; i < MAX_SEQ; i++)
```

```
{ s->is_acked[i] = 0;
```

```
s->timers[i] = 0;
```

```
}
```

```
}
```

```
void receiver_init(Receiver *r)
```

```
{ r->expected_seq_num = 0;
```

```
for(int i = 0; i < MAX_SEQ; i++)
```

```
{ r->received_flags[i] = 0;
```

```
}
```

```
}
```

```
Packet sender_send_packet(Sender *s)
```

```
{
```

```
Packet empty_packet = {-1, ""};
```

```
if(((s->next_seq_to_send - s->window_start + MAX_SEQ)
```

```
% MAX_SEQ) < WINDOW_SIZE)
```

```
{
```

```
if(s->sent_count < MAX_PACKETS) {
```

```
int seq = s->next_seq_to_send;
```

```
Packet p;
```

```
p.seq_num = seq;
```

```
sprintf(p.data, "Data Packet %d", s->sent-
```

```
count + 1);
```

```
s->buffer[seq] = p;
```



```

s->timers[seq] = TIMEOUT;
s->is_acked[seq] = 0;
printf("In [SENDER] Sending Packet # %d (seq %d)\n", s->
    sent_count + 1, seq);
s->next_seq_to_send = (s->next_seq_to_send + 1) %
    MAX_SEQ;
s->sent_count++;
return P;
}

```

```

return empty_packet;
}

```

```

void sender_process_ack(Sender, *s, Ackframe ack)
{

```

```

    int seq = ack.seq_num;

```

```

    if(s->is_acked[seq] == 1)
    {

```

```

        printf("[SENDER] Received duplicate Ack for seq %d.
            Ignoring.\n", seq);
        return;
    }

```

```

    if(s->is_acked[seq] == 0)
    {

```

```

        printf("[SENDER] Received Ack for seq %d. Timer stopped
            .\n", seq);

```

```

        s->is_acked[seq] = 1;

```

```

        s->timers[seq] = 0;

```

```

        while(s->is_acked[s->window_start] == 1)
        {

```

```

            printf("[SENDER] Sliding window forward. Base
                packet (seq %d) acknowledged.\n", s->window_start);

```

```

            s->is_acked[s->window_start] = 0;

```

```

            s->window_start = (s->window_start + 1) % MAX_SEQ;
        }
    }
}

```


Packet sender_check_timeout(Sender *s)

```
Packet empty_packet = {-1, ""};  
int current_seq = s->window_start;  
do {  
    if (s->is_acked[current_seq] == 0 && s->timers  
        [current_seq] > 0)  
    {  
        s->timers[current_seq]--;  
        if (s->timers[current_seq] == 0)  
        {  
            printf("[SENDER] TIMEOUT detected for seq  
                %.d. Retransmitting .... \n", current_seq);  
            s->timers[current_seq] = TIMEOUT;  
            return s->buffer[current_seq];  
        }  
    }  
    current_seq = (current_seq + 1) % MAX_SEQ;  
    if (current_seq == s->next_seq_to_send):  
        break;  
} while(1);  
return empty_packet;
```

Ackframe receiver_process_packet(Receiver *r, Packet p,
int is_lost_flag) {

```
    Ackframe dummy_ack = {-1, -1};  
    if (p.seq_num == -1) return dummy_ack;  
    if (is_lost_flag) {  
        printf("[RECEIVER] Packet (Seq %.d) was LOST in  
            transit. No Ack sent. \n", p.seq_num);  
        return dummy_ack;  
    }  
    printf("[RECEIVER] Received packet (Seq %.d): %.s \n",  
        p.seq_num, p.data);  
    int seq = p.seq_num;
```



```

int expected = r -> expected_seq_num;
int relative_seq = (seq - expected + MAX_SEQ) % MAX_SEQ;

```

```

relative_seq < WINDOW_SIZE) {

```

```

    if (r -> received_flags[seq] == 0)

```

```

    {
        r -> buffer[seq] = p;

```

```

        r -> received_flags[seq] = 1;

```

```

    printf("[RECEIVER] frame Seq %d correctly received and  

    offered (Out-of-Order ok). \n", seq);

```

```

    ckframe ack = {seq, 1};

```

```

    return ack;

```

```

} printf("[RECEIVER] frame Seq %d is outside the  

current window. Ignoring and sending ACK/NACK for base.\n",  

seq); return dummy_ack;

```

```

+ main()

```

```

srand((unsigned int) time(NULL));

```

```

Sender sender;

```

```

Receiver receiver;

```

```

sender_init(&sender);

```

```

receiver_init(&receiver);

```

```

int total_delivered = 0;

```

```

int simulation_cycles = 0;

```

```

const int loss_probability = 20;

```

```

const int ack_loss_probability = 10;

```

```

printf("===== \n");

```

```

printf("Selective Repeat ARQ Simulation \n");

```

```

printf("Window Size: %d, Max Sequence = %d \n", WINDOW_SIZE, MAX_SEQ);

```

```

printf("Total Packets to Send: %d, Loss Probability: %d \n",  

MAX_PACKETS, loss_probability);

```



```

printf("=====\n");
while(total_delivered < MAX_PACKETS) {
    simulation_cycle++;
    printf("\n\n==== CYCLE %d ====\n",
        simulation_cycles);
    while(sender.sent_count < MAX_PACKETS) {
        Packet p = sender_send_packet(&sender);
        if(p.seq_num != -1) {
            Ackframe ack_to_send = receiver_process_packet
            (&receiver, p, is_lost(loss_probability));
            if(ack_to_send.seq_num != -1) {
                if(!is_lost(ack_loss_probability)) {
                    sender_process_ack(&sender, ack_to_send); }
            else { printf("[CHANNEL] Ack for seq %d was Lost.\n",
                ack_to_send.seq_num); } } } }
            Packet retransmit_p = sender_check_timeout(&sender);
            if(retransmit_p.seq_num != -1)
                printf("[SENDER] Retransmitting : Seq %d\n",
                    retransmit_p.seq_num);
            Ackframe ack_to_send = receiver_process_packet(&receiver,
            retransmit_p, is_lost(loss_probability));
            if(ack_to_send.seq_num != -1) {
                if(!is_lost(ack_loss_probability)) {
                    sender_process_ack(&sender, ack_to_send); }
                else { printf("[CHANNEL] ACK for retransmitted
                packet Seq %d was Lost.\n", ack_to_send.seq_num);
                } }
}

```



```

int i = 0;
while (receiver.received_flags[receiver.expected_seq_num]
      == 1) {
    int seq_to_deliver = receiver.expected_seq_num;
    printf("[RECEIVER] ** DELIVERY ** Packet Seq %d :
           %s\n", seq_to_deliver, receiver.buffer[seq_to_deliver],
           data);
    receiver.received_flags[seq_to_deliver] = 0;
    receiver.expected_seq_num = (receiver.expected_seq_num
                                + 1) % MAX_SEQ;
    total_delivered++;
    i++;
    if (i > 0) {
        printf("[RECEIVER] Successfully delivered %d consecutive
               packets. New expected Seq: %d\n", i, receiver.expected
               _seq_num);
    }
    if (total_delivered >= MAX_PACKETS && sender.window_
        start == sender.next_seq_to_send) { break; }
    if (simulation_cycles > 10) {
        printf("[SIMULATION ERROR] Max cycles reached
               (10). Aborting to prevent infinite loop.\n");
        break;
    }
    printf("\n == == == == == == == == == == == == == == \n");
    printf("Selective Repeat ARQ Simulation FINISHED\n");
    printf("Total Packets Delivered: %d\n", total_delivered);
    printf("Total Simulation Cycles: %d\n", simulation_cycles);
    printf("== == == == == == == == == == == == == == \n");
    return 0;
}

```


OUTPUT:-

===== Seq =====

Selective Repeat ARQ Simulation

Window Size: 4, Max Sequence: 8

Total Packets to Send: 10, Loss probability: 20%.

=====

===== CYCLE 1 =====

SENDER] Sending packet #1 (Seq 0)

RECEIVER] Received packet (Seq 0): Data_packet_1

RECEIVER] frame Seq 0 correctly received and buffered
(Out-of-Order ok).

SENDER] Received Ack for Seq 0. Timer stopped.

SENDER] Sliding window forward. Base packet (Seq 0)
acknowledged.

RECEIVER] ** DELIVERY ** Packet Seq 0: Data_packet_1

RECEIVER] Successfully delivered 1 consecutive packets.
New expected Seq: 1

===== CYCLE 2 =====

SENDER] Sending packet #2 (Seq 1)

RECEIVER] Packet (Seq 1) was LOST in transit. No Ack
sent.

===== CYCLE 3 =====

SENDER] Sending packet #3 (Seq 2)

RECEIVER] Received packet (Seq 2): Data_packet_3

RECEIVER] frame Seq 2 correctly received and buffered
(Out-of-order ok).

SENDER] Received Ack for Seq 2. Timer stopped.

===== CYCLE 4 =====

SENDER] Sending Packet #4 (Seq 3)

RECEIVER] Received Packet (Seq 3): Data_packet_4

RECEIVER] frame seq 3 correctly received and buffered

(Out-of-Order OK).

[SENDER] Received Ack for Seq 3. Timer Stopped.

===== CYCLE 5 =====

[SENDER] Sending packet #5 (Seq 4)

[RECEIVER] Packet (Seq 4) was LOST in transit. No Ack sent.

===== CYCLE 6 =====

[SENDER] TIMEOUT detected for Seq 1.
Retransmitting...

[SENDER] Retransmitting : Seq 1

[RECEIVER] Packet (Seq 1) was LOST in transit. No Ack sent.

===== CYCLE 7 =====

===== CYCLE 8 =====

===== CYCLE 9 =====

===== CYCLE 10 =====

[SENDER] TIMEOUT detected for seq 4.
Retransmitting...

[SENDER] Retransmitting : Seq 4

===== CYCLE 11 =====

[SENDER] Sliding window forward. Base packet (Seq 1) acknowledged.

[SENDER] Sliding window forward. Base packet (Seq 4) acknowledged.

=====

Selective Repeat ARQ Simulation finished

Total Packets Delivered : 5

Total Simulation Cycles: 11

=====

Baisakhi Behera
Section :- A
Roll No :- 2505014

ASSIGNMENT-4

Write a C program to implement Go-Back-N ARQ

PROGRAM :-

```

#include<stdio.h>
int main() {
    int total_frames, window_size, lost_frame;
    printf("Enter total number of frames : ");
    scanf("%d", &total_frames);
    printf("Enter window size: \n");
    scanf("%d", &window_size);
    printf("Enter frame number to be lost (1 to %d): ",
        total_frames);
    scanf("%d", &lost_frame);
    printf("\n --- Go back N ARQ Simulation --- \n");
    for(int i=1; i<= total_frames; i++)
    {
        printf("\n Sender sends frames : ");
        for(int j=0; j< window_size && (i+j)<= total_frames; j++)
            printf("%d", i+j);
        printf("\n Receiver: ");
        int success = 1;
        for(int j=0; j< window_size && (i+j)<= total_frames; j++) {
            if(i+j == lost_frame) {
                printf("[Frame %d Lost] \n", lost_frame);
                success = 0;
                break;
            }
            printf("%d", i+j);
        }
        if(!success) {
            printf("Ack Lost! Sender goes back to frame %d \n", i);
        }
        else {
            printf("\n Ack for frames %d to %d \n", i, i+
                window_size-1);
            i += window_size;
        }
    }
}
    
```



```

if(!success)
{
    printf("Retransmitting from frame %d ... \n", D);
}
printf("\n All frames sent successfully! \n");
return 0;

```

OUTPUT:-

Here total number of frames : 5
 Here window size : 4
 Here frame number to be lost (1 to 5) : 8
 -- Go back N ARQ Simulation --
 Sender sends frames : 1 2 3 4
 Receiver : 1 2 3 4
 ACK for frames 1 to 4
 Sender sends frames : 5
 Receiver : 5
 ACK for frames 5 to 8
 All frames sent successfully!

Baisakhi Behera
 Section :- A
 Roll No :- 2505014

ASSIGNMENT-5

Write a C program to implement

- Overview of network simulation tools like Cisco Packet Tracer, GNS3, Wireshark.
- Introduction to basic networking commands. (ping, tracer, ipconfig/ifconfig).

PROGRAM :-

```
#include<stdio.h>
void showTools()
{
    printf("\n === Network Simulation Tools Overview ==\n");
    printf("1. Cisco Packet Tracer : \n");
    printf("- A powerful network simulation tool used for\n designing, configuring, and testing networks \n");
    printf("- Useful to practice CCNA-level labs \n \n");
    printf("2. GNS3 (Graphical Network Simulator 3) : \n");
    printf("- A more advanced network emulator. \n");
    printf("- Supports real Cisco IOS images \n");
    printf("- Used for real world complex network setups.\n \n");
    printf("3. Wireshark : \n");
    printf("- A packet capturing and network protocol analyzer tool. \n");
    printf("- Helps in troubleshooting, monitoring, and studying protocols. \n \n");
}
void showCommands()
{
    printf("\n === Basic Networking Commands Overview === \n");
    printf("1. ping <hostname/IP> : \n");
    printf("- Tests connectivity between your system and another devices \n Example : ping.google.com
```



```

    \n\n");
    printf("2. tracer (Windows) / traceroute (Linux): \n");
    printf("- Shows the path packets take through routers  
to reach destination. \n");
    printf("Example (Windows): tracer google.com \n");
    printf("3. ipconfig (Windows) / ifconfig (Linux): \n");
    printf("- Display IP address, subnet mask, gateway. \n  
- Used to check and configure network interfaces. \n");
    printf("Example (Windows): ipconfig \n");
    printf("Example (Linux): ifconfig \n");
}

int main()
{
    int choice;
    while(1) {
        printf("\n === NETWORK LAB MENU === \n");
        printf("1. Overview of Network Simulation Tools \n");
        printf("2. Basic Networking Commands \n");
        printf("3. Exit \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                showTools();
                break;
            case 2:
                showCommands();
                break;
            case 3:
                printf("Exit \n");
                break;
            default:
                printf("Invalid choice!");
        }
    }
    return 0;
}

```


OUTPUT:-

=== NETWORK LAB MENU ===

1. Overview of Network Simulation Tools
2. Basic Networking Commands
3. Exit

Enter your choice: 2

=== Basic Networking Commands Overview ===

1. ping <hostname/IP>:

- Tests connectivity between your system and another devices.

Example: ping google.com

2. traceret (Windows) / traceroute (Linux):

- Shows the path packets take through routers to reach destination.

Example (Windows): traceret google.com

Example (Linux): traceroute google.com

=== NETWORK LAB MENU ===

1. Overview of Network Simulation Tools
2. Basic Networking Commands
3. Exit

Enter your choice: 3

Exit

Name:- Baisakhi Behera
RollNo:- 2505014

ASSIGNMENT- 6

Write a C program for setting up a simple peer-to-peer network.

o Configuring IP addresses and subnet masks.

o Testing network connectivity using ping and tracer.

PROGRAM:- #include <stdio.h>

int main() {

printf("Setting up a simple Peer-to-peer Network:\n");

printf("1. Connect two computers using a crossover cable or LAN cable\n");

printf("Configuring IP Addresses and Subnet Masks:\n");

printf("IP of PC1: 192.168.1.1\n");

printf("Subnet Mask: 255.255.255.0\n");

printf("IP of PC2: 192.168.1.2\n");

printf("Subnet Mask: 255.255.255.0\n\n");

printf("Testing Network Connectivity: \n");

printf("Command: ping 192.168.1.2\n");

printf("Command: tracer 192.168.1.2\n");

return 0;

}

Output:- Setting up a Simple Peer-to-Peer Network
1. Connect two computers using a crossover cable or LAN cable.

Configuring IP Addresses and Subnet Masks:

IP of PC1: 192.168.1.1

Subnet Mask: 255.255.255.0

IP of PC2: 192.168.1.2

Subnet Mask: 255.255.255.0

Testing Network Connectivity:

Command: ping 192.168.1.2

Command: tracer 192.168.1.2

Baisakhi Behera
Roll No:- 2505014

ASSIGNMENT- 7

Write a C program for Error Detection and Correction.

- o Implementing CRC error detection.
- o Simulating error correction mechanisms.

PROGRAM:-

```
#include<stdio.h>
#include<string.h>
#define KEY "10011"
#define KEY_SIZE 5
#define CRC_BITS (KEY_SIZE - 1)

void crc_division(char *data, const char *key)
{
    int data_len = strlen(data);
    for(int i=0; i<= data_len - KEY_SIZE; i++)
    {
        if(data[i] == '1')
        {
            for(int j=0; j<KEY_SIZE; j++)
            {
                data[i+j] = (data[i+j] == key[j]) ? '0' : '1';
            }
        }
    }
}

int main()
{
    char data[] = "11010110";
    char key[] = KEY;
    int data_len = strlen(data);
    char padded_data[30];
    strcpy(padded_data, data);
    for(int i=0; i< CRC_BITS; i++)
    {
        strcat(padded_data, "0");
    }
    char temp_data[30];
    strcpy(temp_data, padded_data);
```



```

crc_division(temp_data, key);
char crc_remainder[CRC_BITS+1];
strcpy(crc_remainder, temp_data + data_len, CRC_BITS);
crc_remainder[CRC_BITS] = '\0';

char transmitted_codeword[30];
strcpy(transmitted_codeword, data);
strcat(transmitted_codeword, crc_remainder);

printf("CRC ERROR DETECTION SIMULATION \n");
printf("Data : %s, Generator: %s \n", data, KEY);
printf("Generated CRC: %s \n", crc_remainder);
printf("Transmitted Codeword : %s \n", transmitted_codeword);

char received_data1[30];
strcpy(received_data1, transmitted_codeword);
strcpy(temp_data, received_data1);
crc_division(temp_data, key);

char syndrome_1[CRC_BITS+1];
strcpy(syndrome_1, temp_data + data_len, CRC_BITS);
syndrome_1[CRC_BITS] = '\0';

printf("Received Codeword : %s \n", received_data1);
printf("Final Remainder : %s -> ", syndrome_1);
printf("strcmp(syndrome_1, \"0000\") == 0 ? \"No Error Detected \n\" : \"Error Detected \n\");

```

OUTPUT:- CRC ERROR DETECTION SIMULATION
 Data : 11010110, Generator: 10011
 Generated CRC : 0010
 Transmitted Codeword: 110101100010
 Receiving Codeword : 110101100010
 Final Remainder : 0000 -> No Error Detected

Baishakhi Behara
 Roll No:- 2505014

ASSIGNMENT-8

Write a C program for Medium Access Control

- o Configuring and analyzing Ethernet networks,
- o Setting up and testing Wireless LAN (WLAN) connections,
- o Exploring Bluetooth network configurations,

PROGRAM:-

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    printf(" Network Interface Configuration Analysis\n");
    printf("1. Ethernet/Wired Network Information : \n");
    printf("> Command: ifconfig);
    printf("\n --- Section End --- \n");
    printf("2. Wireless LAN Connection : \n");
    printf("> Command: iwconfig);
    printf("\n --- Section End --- \n");
    printf("3. Bluetooth Network Configuration: \n");
    printf("> Command: hciconfig);
    printf("Execution Completed\n");
    return 0;
}
```

OUTPUT:-

Network Interface Configuration Analysis

1. Ethernet / Wired Network Information :

> Command : ifconfig

eth0: flags = 4163 < UP, BROADCAST, RUNNING, MULTICAST >
mtu 1500

inet 192.168.1.100 netmask 255.255.255.0

broadcast 192.168.1.255

inet6 fe80::5054:ff:fe00:1 prefixlen 64 scopeid

0x20 <link>

ether 00:01:02:AA:BB:CC txqueuelen 1000

(Ethernet)

RX packets 1234 bytes 150000 (150.0 KB)

..


```
lo : flags = 73 <UP, LOOPBACK, RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet ::1 prefixlen 128 scopeid 0x10 <host>
    ...
```

--- Section End ---

2. Wireless LAN Connection :

>Command: iwconfig

wlan0 IEEE 802.11 ESSID: "MyHomeWiFi"

Mode: Managed Frequency: 2.412 GHz Access point:

DS: CB: 8A: 2f: 3D: 4E

Bit Rate = 54 Mbit/s Tx-power = 20 dBm

Retry short limit: 7 RTS thr: off fragment thr: off

Power Management: off

Link Quality = 70/70 Signal level = -40 dBm

Rx Invalid nwid: 0 Rx invalid crypt: 0 Rx invalid
frag: 0

Tx excessive retries: 0 Invalid misc: 0 Missed beacon: 0

--- Section End ---

3. Bluetooth Network Configuration

>Command: hciconfig

hci0 : Type: Primary Bus: USB

BD Address: 00:0A:0B:0C:0D:0E ACL MTU: 1021.8

SCO MTU: 64:1

UP RUNNING

Rx Bytes: 1440 acl: 0 sco: 0 events: 48 errors: 0

Tx Bytes: 308 acl: 0 sco: 0 commands: 48 errors: 0

--- Section End ---

Baisakhi Behera
Roll No: - 2505014
Section: A

ASSIGNMENT-9

Write a C program for Network Devices Configuration.

- o Setting up and configuration of switches, routers and gateways.

- o Understanding the use of repeaters, hub, bridges in a network.

PROGRAM:- #include <stdio.h>
#include <stdlib.h>

```
int main() { printf("Network Device Configuration check\n");
```

```
printf("1. IP Routing Table : \n");
```

```
printf("> Command: route -n");
```

```
printf("\n --- Section End --- \n");
```

```
printf("2. DNS / Name Server Configuration : \n");
```

```
printf("> Command : cat /etc/resolv.conf");
```

```
printf("\n --- Section End --- \n");
```

```
printf("3. Switches, Routers and Hardware Devices : \n");
```

```
printf("> Direct Configuration of Switches / Routers  
requires administrative protocols or specialized vendor  
tools, and cannot be done with simple C commands. \n");
```

```
printf("Execution Completed \n");
```

```
return 0;
```

```
}
```

OUTPUT:- Network Device Configuration Check

1. IP Routing Table :

> Command : route -n

kernel IP routing table

Destination	Gateway	Genmask	flags	Metric	Ref
-------------	---------	---------	-------	--------	-----

Use Iface

0.0.0.0	192.168.1.1	0.0.0.0	UG	100	0	eth0
---------	-------------	---------	----	-----	---	------

192.168.1.0	0.0.0.0	255.255.255.0	U	100	0	eth0
-------------	---------	---------------	---	-----	---	------

169.254.0.0	0.0.0.0	255.255.255.0	U	1000	0	0weth0
-------------	---------	---------------	---	------	---	--------

--- Section End ---

2. DNS / Name Server Configuration :

> Command: cat /etc/resolv.conf

Generated by NetworkManager
search Localdomain

nameserver 192.168.1.1

nameserver 8.8.8.8

--- Section End ---

3. Switches, Routers, and Hardware Devices :

> Direct configuration of Switches / Routers requires administrative protocols or specialized vendor tools, and cannot be done with simple c Commands.

Execution Completed

Baisakhi Behera

Roll No:- 2505014

Section:- A